

U3 · Aula 4 — Testando PWAs

Service Workers, Offline Mode e Lighthouse CI

Prof. Jackson Smith Moisés Matias

PUC Minas IEC · Qualidade Web e Mobile (EAD)

Roteiro da aula

- **Parte 1 — Service Worker:** lifecycle, cache strategies, como esperar certo
- **Parte 2 — Instalação e offline:** manifest, install prompt, Background Sync
- **Parte 3 — Lighthouse CI:** budgets bloqueando merge
- **Parte 4 — Prática guiada:** PWA offline do CineFav Web

Depois desta aula: screencast — eu resolvendo os specs passo a passo.

Objetivos de aprendizagem

- **Explicar** o lifecycle do SW (install → waiting → activate → controlling)
- **Testar** modo offline de verdade: cache populado → `setOffline` → reload
- **Validar** manifest e install prompt de uma PWA
- **Configurar** Lighthouse CI com budgets que bloqueiam merge
- **Completar** o spec de PWA do CineFav Web + rodar Lighthouse local

Parte 1 — Service Worker

Lifecycle · cache strategies · como esperar certo

Desafios de testar Service Workers

O SW roda em **thread separada** do main thread. Para testar:

1. Forçar registro (automático só em produção)
2. Aguardar estado `activate`
3. Conhecer o estado do cache
4. Simular condições de rede

A maioria dos bugs de SW aparece em **transições de estado** (install → waiting → activate → controlling).

Aguardar SW pronto — o jeito que NÃO flaka

```
test('Service Worker controla a página', async ({ page }) => {
  await page.goto('/');

  // ⚠ serviceWorker.ready resolve com o worker ainda em "activating"!
  // expect.poll re-consulta até o estado terminal:
  await expect
    .poll(() =>
      page.evaluate(async () => {
        const reg = await navigator.serviceWorker.getRegistration();
        return reg?.active?.state;
      }),
    )
    .toBe('activated');
});
```

E pra navegação offline: "activated" ainda não basta — quem responde a navegação é o SW **controlando** a página:

```
waitForFunction(() => navigator.serviceWorker.controller !== null)
```

Cache-first strategy — testar

```
test('cache-first serve do cache quando offline', async ({ page, context }) => {  
  // Visita inicial – popula o cache  
  await page.goto('/');  
  await page.waitForLoadState('networkidle');  
  
  // Simular perda total de conectividade  
  await context.setOffline(true);  
  
  // Recarregar – deve servir do Service Worker cache  
  await page.reload();  
  
  await expect(page.getByText('Welcome')).toBeVisible();  
  // UI deve indicar modo offline  
  await expect(page.getByTestId('offline-banner')).toBeVisible();  
});
```

SW lifecycle — skipWaiting test

```
test('SW updates when new version deployed', async ({ page }) => {  
  // Cenário: nova versão SW disponível  
  await page.goto('/');  
  
  await page.evaluate(() => {  
    return new Promise<void>((resolve) => {  
      navigator.serviceWorker.addEventListener('controllerchange', () => resolve());  
      navigator.serviceWorker.ready.then((reg) => {  
        // Simula update  
        reg.update();  
      });  
    });  
  });  
  
  // Verificar que cliente foi assumido por novo SW  
});
```

Cenários intermediários (waiting state) são onde bugs aparecem.

Parte 2 — Instalação e offline

Manifest · install prompt · offline · Background Sync

PWA manifest — validação

```
test('manifest.json é válido', async ({ page }) => {
  const response = await page.goto('/manifest.json');
  expect(response?.status()).toBe(200);

  const manifest = await response?.json();

  expect(manifest.name).toBeTruthy();
  expect(manifest.short_name).toBeTruthy();
  expect(manifest.start_url).toBe('/');
  expect(manifest.display).toMatch(/standalone|fullscreen/);
  expect(manifest.icons.find((i: any) => i.sizes === '192x192')).toBeDefined();
  expect(manifest.icons.find((i: any) => i.sizes === '512x512')).toBeDefined();
});
```

PWA — install prompt simulation

```
test('install prompt fires on supported browsers', async ({ page }) => {
  let installPromptFired = false;

  await page.exposeFunction('onBeforeInstallPrompt', () => {
    installPromptFired = true;
  });

  await page.addInitScript(() => {
    window.addEventListener('beforeinstallprompt', () => {
      (window as any).onBeforeInstallPrompt();
    });
  });

  await page.goto('/');
  // Trigger criteria do browser

  expect(installPromptFired).toBe(true);
});
```

Modo offline — teste completo

```
test('offline mode: conteúdo cacheado + banner + fila de sync', async ({
  page,
  context,
}) => {
  await page.goto('/mensagens');
  await page.waitForLoadState('networkidle');

  await context.setOffline(true);
  await page.reload();

  // Conteúdo principal disponível via cache
  await expect(page.getByText('Suas mensagens')).toBeVisible();

  // Banner de offline visível
  await expect(page.getByTestId('offline-banner')).toBeVisible();

  // Enviar mensagem offline — deve ficar em fila
  await page.getByPlaceholder('Digite uma mensagem').fill('Olá offline!');
  await page.getByRole('button', { name: 'Enviar' }).click();
  await expect(page.getByText('Pending sync')).toBeVisible();
});
```

Background Sync verification

```
test('queues message for sync when offline', async ({ page, context }) => {
  await page.goto('/messages');
  await context.setOffline(true);

  await page.fill('[name=message]', 'Olá!');
  await page.click('button[type=submit]');

  // Mensagem fica em estado "pending sync"
  await expect(page.getByText('Pending sync')).toBeVisible();

  // Voltar online
  await context.setOffline(false);

  // Background Sync deve disparar
  await page.evaluate(async () => {
    const reg = await navigator.serviceWorker.ready;
    return reg.sync.register('flush-outbox');
  });

  await page.waitForTimeout(2000);
  await expect(page.getByText('Sent')).toBeVisible();
});
```

Parte 3 — Lighthouse CI

Budgets de performance bloqueando merge

Lighthouse CI — configuração

```
npm install -g @lhci/cli@0.13.x
```

```
{
  "ci": {
    "collect": {
      "url": ["http://localhost:3000/", "http://localhost:3000/checkout"],
      "numberOfRuns": 3
    },
    "assert": {
      "assertions": {
        "categories:performance": ["error", { "minScore": 0.9 }],
        "categories:pwa": ["error", { "minScore": 0.9 }],
        "categories:accessibility": ["error", { "minScore": 0.9 }],
        "largest-contentful-paint": ["error", { "maxNumericValue": 2500 }],
        "first-contentful-paint": ["warn", { "maxNumericValue": 2000 }]
      }
    }
  }
}
```

Performance budgets

```
{
  "ci": {
    "assert": {
      "assertions": {
        "resource-summary:script:size": [
          "error", { "maxNumericValue": 250000 }
        ],
        "resource-summary:image:size": [
          "error", { "maxNumericValue": 500000 }
        ],
        "resource-summary:stylesheet:size": [
          "error", { "maxNumericValue": 50000 }
        ]
      }
    }
  }
}
```

PR que aumenta bundle JS além de 250KB → **CI falha, merge bloqueado**. Evita *death by a thousand cuts*.

Lighthouse CI no GitHub Actions

```
name: Lighthouse CI
on: [pull_request]

jobs:
  lighthouse:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 22 }
      - run: npm ci && npm run build
      - run: npm install -g @lhci/cli@0.13.x
      - run: lhci autorun
    env:
      LHCI_GITHUB_APP_TOKEN: ${ secrets.LHCI_GITHUB_APP_TOKEN }
```

Resulta em comment no PR com score e link para report. Falha bloqueia merge.

Pitfalls comuns

- ✗ **Lighthouse CI com 1 run** — variação alta; configure `numberOfRuns: 3`
- ✗ **Budget muito apertado** — bloqueia PRs legítimos por variação de medição
- ✗ **Não limpar cache entre testes de SW** — test pollution (estado de cache vaza)
- ✗ **Testar SW sem aguardar `ready`** — race condition, falha intermitente
- ✗ `setOffline(true)` **sem popular cache antes** — teste testa a falha do teste, não o offline

Resumo

- **SW testing:** aguardar `ready` , limpar cache entre testes, usar `setOffline`
- **Manifest validation:** checar campos obrigatórios + ícones 192/512px
- **Offline mode:** popular cache → offline → recarregar → verificar conteúdo + fila
- **Lighthouse CI:** 3 runs, budgets de performance, bloqueio de merge
- **Budgets:** JS < 250KB, imagens < 500KB — impõem disciplina no PR

Parte 4 — Prática guiada

PWA offline do CineFav Web + Lighthouse local

A prática — 05-pwa-offline.spec.ts

O CineFav Web é uma PWA (vite-plugin-pwa + Workbox):

```
cd exercicios/02-lab-web-pwa-playwright/pratica
ls tests/e2e      # confirma: 05-pwa-offline.spec.ts está aí
npm run test:e2e # roda contra o BUILD – SW não existe no dev server!
```

- Prática 1 (resolvida em parte): SW ativo com `expect.poll`
- Prática 2: validar o manifest (name, display, ícones)
- Prática 3 (● desafio): a sequência offline completa

Prática 3 — o teste offline, passo a passo

```
// 1. visita online (SW instala e precacheia)
// 2. espere o SW CONTROLAR a página:
await page.waitForFunction(() => navigator.serviceWorker.controller !== null);
// 3. derrube a rede DO CONTEXT0:
await context.setOffline(true);
// 4. recarregue – app shell + catálogo vêm do cache:
await page.reload();
// 5. movielist-grid visível + offline-banner visível
```

- Cada passo é um TODO no scaffold — a ordem importa
- Pular o passo 2 = `ERR_INTERNET_DISCONNECTED` intermitente (pitfall real)

Prática Lighthouse — budgets no CineFav

```
npm run build
npm run lighthouse    # lhci autorun com lighthouseconfig.json
```

- `lighthouseconfig.json` já traz **3 budgets**: FCP < 2s, LCP < 2.5s, TBT < 300ms
 - minScore 0.9 em performance / a11y / best-practices
- Rode, leia o report, depois **aperte um budget até falhar** — entender o bloqueio é o objetivo
- No lab: print/log da run com budgets passa a fazer parte da entrega

Como acompanhar

- **Screencast** — vídeo na sequência desta unidade
- **Repo da prática** — link no Canvas, specs-guia prontos
- **SW travado?** — DevTools → Application → Service Workers → Unregister e rode de novo

Bons estudos!

 jackson.96@gmail.com

 linkedin.com/in/3jacksonsmith

 github.com/jacksonsmith